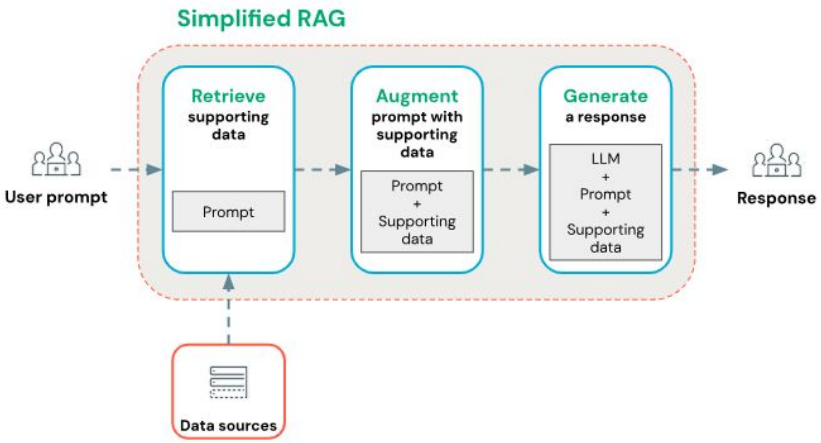
Python 200
Week 6: AI Augmentation

CODE THE DREAM

Knowledge Augmentation for LLMs

- LLMs – text generators, but they hallucinate. Guess, misremember or invent facts.
- Solutions to make LLMs smarter: prompt engineering, fine-tuning, and RAG.
- Prompt engineering
  - Paste facts directly into the prompt.
  - Simple and fast for small info.
  - Long prompts are costly and get ignored.



- Fine tuning
  - Retrain the model on custom data.
  - Make the model adopt to a style.
  - Expensive and require many examples.

- RAG
  - Retrieve relevant info from database and inject into prompt.
  - Updatable, scalable, no retraining needed.

- Lesson path: keyword retrieval -> semantic retrieval -> LlamaIndex.

Keyword-based RAG - Retrieval

- It is useful for learning, but brittle in real-world use.
- Build a Chat bot with RAG.
- Retrieves full documents, not chunked passages.
- Extract PDF text into a searchable corpus.
- Retrieving the best suited document for the query to augment.
  - Remove irrelevant words
  - Store reduced words as set
  - Overlap score between the sets and the each document.
  - Based on no of common words picks the best document.

```
def extract_text_from_pdf(pdf_path):
    reader = PdfReader(pdf_path)
    text = []
    for page in reader.pages:
        page_text =
page.extract_text()
        if page_text:

text.append(page_text.strip())
    return "\n".join(text)
```

```
def simple_keyword_retrieval(query, documents):
   import string
    stopwords = ["a", "an", "the"]
    translator = str.maketrans("", "",
string.punctuation)
    query_words = {
        w.translate(translator)
        for w in query.lower().split()
        if w not in stopwords
    }
    scores = []
    for name, content in documents.items():
        content_words = {
            w.translate(translator)
            for w in content.lower().split()
            if w not in stopwords
        }
        overlap = query_words & content_words
        score = len(overlap)
        scores.append((score, name, content))
    scores.sort(reverse=True)
    best = next(((name, content) for score, name,
content in scores if score > 0), None)
    if best:
        return [best]
    else:
        return [("None found", "No relevant
content.")]
```

Keyword RAG: Generation

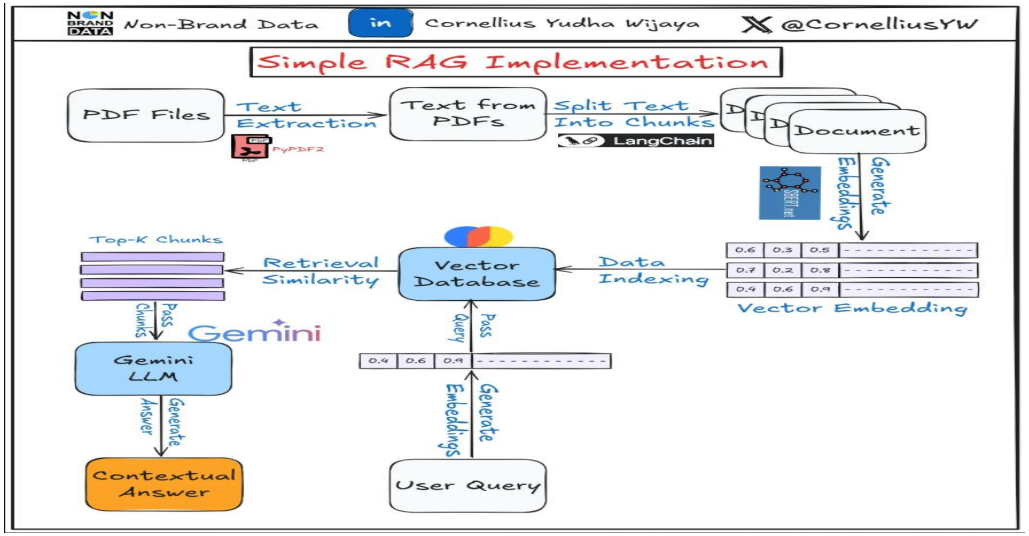
- Retrieved context is appended to the user question before generation.
- This makes the retrieval step explicit and easy to inspect.
- Limitation: exact word overlap misses semantic matches and synonyms.

```
def ask_llm(query, context, with_rag):
    if with_rag:
        prompt = (
           f"Use the following context to
answer\n"

f"Context:\n{context.strip()}\n\n"
           f"Question: {query}\nAnswer:"
        )
    else:
        prompt = f"Question:
{query}\nAnswer:"

    response =
client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user",
                   "content": prompt}],
        temperature=0.2
    )
```

Semantic RAG: Chunking

- Industry standard RAG.
- Documents are broken into smaller overlapping chunks. Convert to vector embedding's.
- Smaller units improve retrieval quality and reduce wasted prompt tokens.
- In this lesson: chunk_size = 900 and overlap = 150.

```
def simple_chunks(text: str, chunk_size=900,
overlap=150):
    chunks = []
    i = 0
    L = len(text)
    while i < L:
        chunk = text[i:i+chunk_size]
        end = chunk.rfind(". ")
        if end > int(chunk_size * 0.6):
            chunk = chunk[:end+1]
        chunk = chunk.strip()
        if chunk:
            chunks.append(chunk)
        i += max(1, len(chunk) - overlap)
    return chunks
```



Semantic RAG: Embeddings + Retrieval

- Each chunk is converted into a dense vector embedding.
- Similarity is measured in embedding space instead of raw text space.
- Top-k chunks become the grounded context for the final answer.
- This is much closer to modern industry RAG pipelines.

```
def embed_texts(texts,
model="text-embedding-3-small"):
    vecs = []
    for i in range(0, len(texts), 128):
        batch = texts[i:i+128]
        resp = client.embeddings.create(
            model=model, input=batch)
        vecs.extend([np.array(e.embedding,
                    dtype="float32")
                    for e in resp.data])
    return np.vstack(vecs)
```

```
def retrieve(query, k=3,
model="text-embedding-3-small"):
    q_emb = embed_texts([query],
model=model)
    faiss.normalize_L2(q_emb)
    D, I = index.search(q_emb, k)
    meta = id2meta[int(idx)]
        hits.append({"score": float(score),
**meta})
    return hits
```

Semantic RAG: Augment And Generate

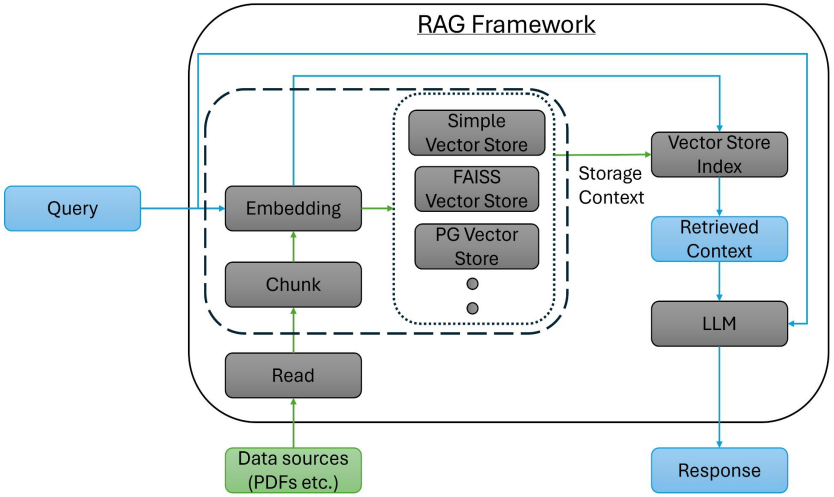
- Retrieved chunks are concatenated into one grounded prompt.
- The prompt tells the model to use only the provided context.
- If the answer is missing from the retrieved passages, the model should say so.
- This is one of the main safeguards against hallucination.

```
def ask_llm(query, contexts, use_rag,
            model="gpt-4o-mini", temperature=0.2):
    ctx = "\n\n---\n\n".join(c["text"] for c in
contexts)
    if use_rag:
        prompt = (
            "Use ONLY the provided context to
answer.\n"
            "If the answer is not in the context, say
so.\n\n"
            f"Context:\n{ctx}\n\n"
            f"Question: {query}\nAnswer:"
        )
    else:
        prompt = query
        resp = client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content":
prompt}],
        temperature=temperature
    )
     return resp.choices[0].message.content.strip()
```

LlamaIndex: Read, Chunk, and Store

- LlamaIndex wraps reading, chunking, embedding, and indexing into a simpler API.
- SimpleDirectoryReader loads source files directly from the folder.
- VectorStoreIndex.from_documents() builds the semantic index automatically.
- Result: much less boilerplate than the custom implementation.

```
from llama_index.core import (
    SimpleDirectoryReader, VectorStoreIndex)

# Load documents directly from PDFs
docs = SimpleDirectoryReader(
    "brightleaf_pdfs").load_data()

# Build a vector index automatically
index =
VectorStoreIndex.from_documents(docs)

print(type(index._vector_store).__name__)
# SimpleVectorStore
```



LlamaIndex: Query in Four Lines

- The query engine handles retrieval and response generation.
- similarity_top_k controls how much supporting context is passed in.
- Source nodes can still be inspected for transparency and evaluation.
- Same RAG idea as before, but packaged for faster development.

```
query_engine = index.as_query_engine(
    similarity_top_k=3)

questions = [
    "What is BrightLeaf Solar's mission?",
    "How did profits change between 2023 and
2024?",
    "Which partner joined most recently?"
]

for q in questions:
    print(f"\nQ: {q}")
    response = query_engine.query(q)
    print("A:", response)
    for node_with_score in
response.source_nodes:
        print(node_with_score.score)
```

LlamaIndex: RAG Evaluation

- Evaluates RAG by computing metrics for retrieval process or the response generation process.

- Relevancy: Evaluates retrieval.
  - Response is relevant to the query and prevents off-topic rambling.
- Faithfulness: Evaluates generation.
  - Response aligns with the retrieved contexts and avoids hallucinations.

- Implemented in LlamaIndex using the FaithfulnessEvaluator and RelevancyEvaluator tools.

- Faithfulness Evaluation: 1.0
- Relevancy Result: 1.0

```
from llama_index.llms.openai import OpenAI
from llama_index.core.evaluation import
FaithfulnessEvaluator, RelevancyEvaluator

llm = OpenAI(model="gpt-4o-mini",
temperature=0.2)

faithfulness_evaluator =
FaithfulnessEvaluator(llm=llm)
relevancy_evaluator =
RelevancyEvaluator(llm=llm)

q = "What is BrightLeaf Solar's mission?"
response = query_engine.query(q)

# Evaluate faithfulness and relevancy
faithfulness_result =
faithfulness_evaluator.evaluate_response(query=q
, response=response)
print("Faithfulness Evaluation: " +
str(faithfulness_result.score))

relevancy_result =
relevancy_evaluator.evaluate_response(query=q,
response=response)
print("Relevancy Result: " +
str(relevancy_result.score))
```

Week 6 Wrap-up

- Prompt engineering is simple, but prompt length and cost become limiting factors.
- Fine-tuning changes model behavior, but it is heavier, slower, and more rigid.
- Keyword RAG teaches the retrieval logic; semantic RAG improves relevance with embeddings.
- LlamaIndex packages the same steps with far less boilerplate.
- Key takeaway: retrieve the right context before asking the model to answer.